`Terraform-02`.

1). Watch the Terraform-02 video.

Lab 1: Terraform Providers, Local File, Random Pet and State File

Operating System: Ubuntu Linux (AWS EC2)
Terraform Version: 1.17.0
Working Directory: `/home/ubuntu/terraform-02/lab1`

---

1. Objective

The objective of this task is to understand and implement Terraform using multiple providers.

We will:

- Configure Local and Random providers.
- Create a text file using Terraform.
- Generate a random pet name.
- Understand Terraform initialization and execution.
- Verify the created resources using Terraform state.

2. Prerequisites

- Ubuntu EC2 instance
- Terraform installed
- Terminal access
- Internet connectivity to download Terraform providers

Step 1: Verify Terraform Installation

Executed the following commands:

terraform version
git --version
sudo systemctl is-active jenkins

Output:

Terraform v1.17.0

on linux_amd64

git version 2.43.0
active

Result: Terraform, Git and Jenkins were available on the EC2 instance.

3. Create Project Directory

Step 2: Create a Separate Directory for Terraform-02

mkdir -p ~/terraform-02/lab1
cd ~/terraform-02/lab1
pwd

Output:

/home/ubuntu/terraform-02/lab1

Purpose: Maintain Terraform-02 separately without affecting the previous Terraform-01 assignment.

4. Create Terraform Configuration File

Step 3: Create main.tf

```
cat > main.tf <<'EOF'
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
    random = {
      source = "hashicorp/random"
    }
  }
}

resource "local_file" "my_pet" {
  filename = "${path.module}/pets.txt"
  content  = "My cat name is Cat"
}

resource "random_pet" "my_pet" {
```

```
  prefix    = "Mr"
  separator = "."
  length    = 1
}

output "pet_name" {
  value = random_pet.my_pet.id
}
EOF
```

Explanation:

- `required_providers` – Declares which providers are required.
- `local_file` – Creates a local text file.
- `random_pet` – Generates a random name.
- `output` – Displays the generated pet name.

Step 4: Verify Configuration

```
ls -l
cat main.tf
```

Result: The `main.tf` file was created and the configuration was verified.

5. Initialize Terraform

Step 5: Download Provider Plugins

```
terraform init
```

Output:

```
Installing hashicorp/local v2.9.1...
Installed hashicorp/local v2.9.1

Installing hashicorp/random v3.9.1...
Installed hashicorp/random v3.9.1

Terraform has been successfully initialized!
```

Purpose: Downloads the required providers and prepares the working directory.

Terraform also creates `.terraform/` and `.terraform.lock.hcl`.

6. Format and Validate Configuration

Step 6: Format Terraform Code

terraform fmt

Purpose: Formats Terraform configuration according to Terraform's standard style.

Step 7: Validate Configuration

terraform validate

Output:

Success! The configuration is valid.

Result: The Terraform configuration was syntactically valid.

7. Preview Infrastructure Changes

Step 8: Execute Terraform Plan

terraform plan

Output:

local_file.my_pet will be created

random_pet.my_pet will be created

Plan: 2 to add, 0 to change, 0 to destroy.

Explanation: Terraform planned to create two resources: a local file and a random pet name.

8. Execute Terraform Configuration

Step 9: Run Terraform Apply

terraform apply

When prompted:

Enter a value: yes

Actual output:

random_pet.my_pet: Creation complete
[id=Mr.quagga]

local_file.my_pet: Creation complete

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

pet_name = "Mr.quagga"

Result: Both Terraform resources were successfully created.

9. Verify Terraform Resources

Step 10: Check Terraform State

terraform state list

Output:

local_file.my_pet
random_pet.my_pet

Explanation: Terraform state tracks both resources created by the configuration.

Step 11: Verify Random Pet Output

terraform output pet_name

Output:

"Mr.quagga"

Step 12: Verify Local File Content

cat pets.txt

Output:

My cat name is Cat

Result: The local file was created with the expected content.

10. Understanding the Terraform State File

After executing `terraform apply`, Terraform creates a local state file named:

terraform.tfstate

The state file stores information about resources managed by Terraform, including their IDs and attributes.

Useful commands:

terraform state list
terraform state show local_file.my_pet
terraform state show random_pet.my_pet

Important: Avoid manually editing the Terraform state file. Do not commit local state files to a public Git repository.

11. Final Verification

Verification	Result
Terraform installed	Successful
Providers initialized	Successful
Configuration validated	Successful
Terraform plan	2 resources to add
Terraform apply	Successful
Local file created	Successful
Random pet generated	Mr.quagga
Terraform state verification	2 resources tracked

12. Learning Outcomes

Through this practical task, I learned:

1. How to declare and initialize multiple Terraform providers.
2. How to create a local file using Terraform.
3. How to generate random names using the Random provider.
4. The purpose of `terraform init`, `fmt`, `validate`, `plan`, and `apply`.
5. How Terraform records and manages resources using its state file.

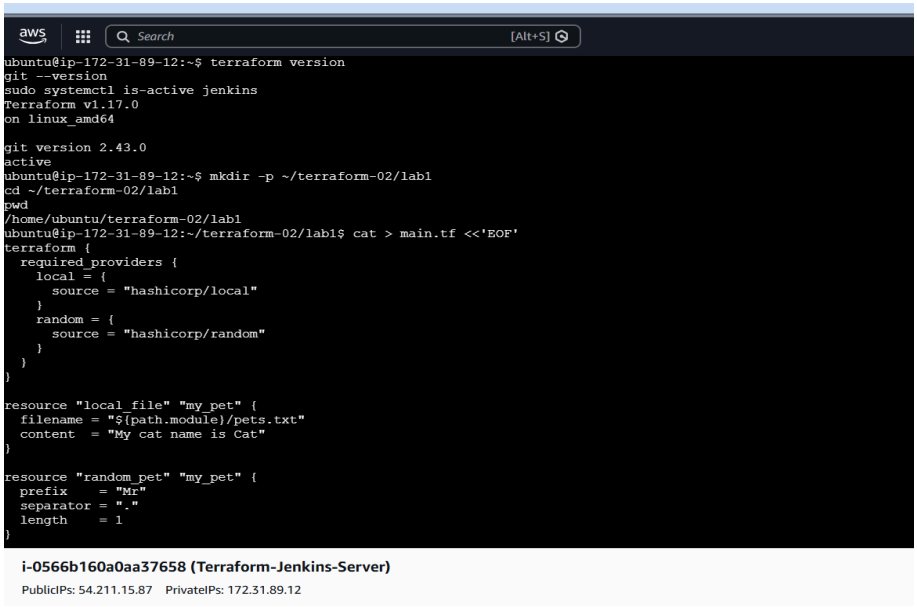
6. How to verify Terraform-created resources using state commands and outputs.

13. Conclusion

Successfully completed Terraform-02 Lab 1 using the Local and Random providers on an Ubuntu EC2 instance.

Terraform created `pets.txt` with the content `My cat name is Cat` and generated the random pet name `Mr.quagga`.

Both resources were verified using Terraform state and output commands.






```
  length    = 1
}

output "pet_name" {
  value = random_pet.my_pet.id
}
ubuntu@ip-172-31-89-12:~/terraform-02/lab1$ terraform init
terraform fmt
terraform validate
terraform plan
Initializing the backend...

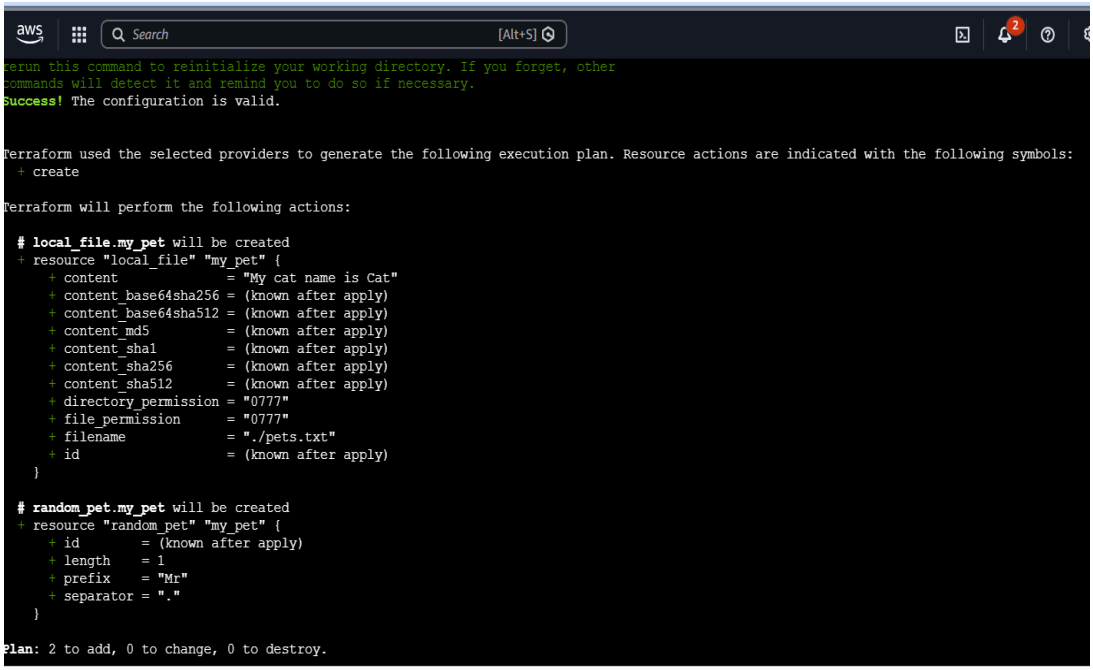
Initializing provider plugins...
- Finding latest version of hashicorp/local...
- Finding latest version of hashicorp/random...
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)
- Installing hashicorp/random v3.9.1...
- Installed hashicorp/random v3.9.1 (signed by HashiCorp)

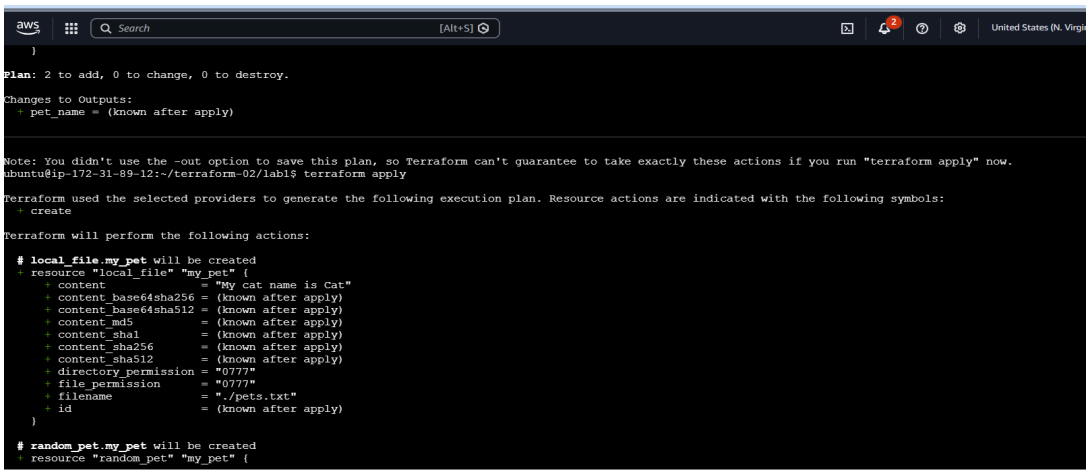
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

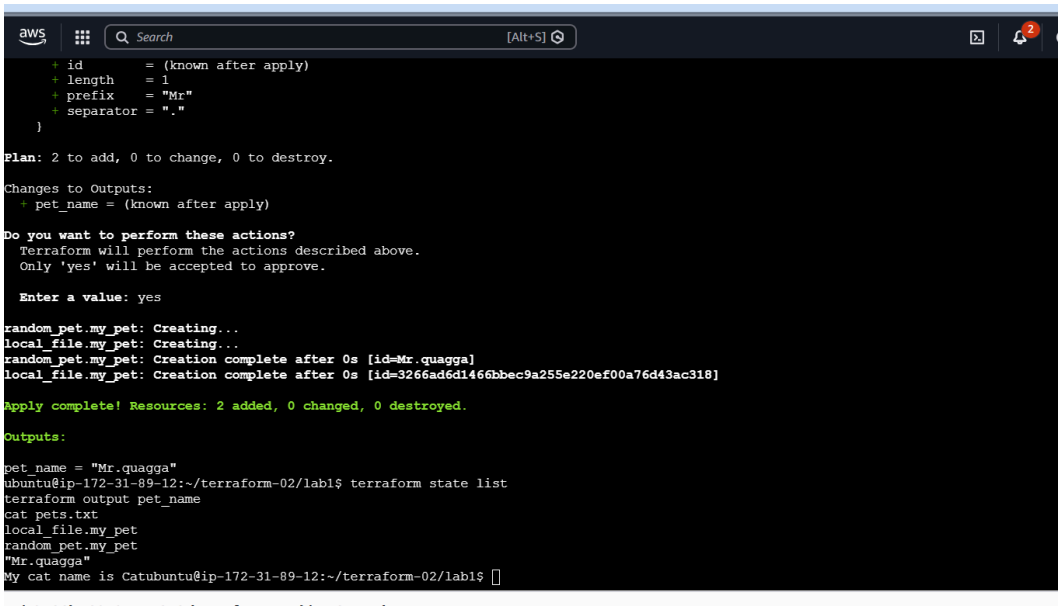
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```


aws
Search
[Alt+S]
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
Success! The configuration is valid.
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
local_file.my_pet will be created
+ resource "local_file" "my_pet" {
+ content = "My cat name is Cat"
+ content_base64sha256 = (known after apply)
+ content_base64sha512 = (known after apply)
+ content_md5 = (known after apply)
+ content_sha1 = (known after apply)
+ content_sha256 = (known after apply)
+ content_sha512 = (known after apply)
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "./pets.txt"
+ id = (known after apply)
}
random_pet.my_pet will be created
+ resource "random_pet" "my_pet" {
+ id = (known after apply)
+ length = 1
+ prefix = "Mr"
+ separator = "."
}
Plan: 2 to add, 0 to change, 0 to destroy.


aws
Search
[Alt+S]
United States (N. Virgi
}
Plan: 2 to add, 0 to change, 0 to destroy.
Changes to Outputs:
+ pet_name = (known after apply)
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
ubuntu@ip-172-31-89-12:~/terraform-02/lab1$ terraform apply
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
local_file.my_pet will be created
+ resource "local_file" "my_pet" {
+ content = "My cat name is Cat"
+ content_base64sha256 = (known after apply)
+ content_base64sha512 = (known after apply)
+ content_md5 = (known after apply)
+ content_sha1 = (known after apply)
+ content_sha256 = (known after apply)
+ content_sha512 = (known after apply)
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "./pets.txt"
+ id = (known after apply)
}
random_pet.my_pet will be created
+ resource "random_pet" "my_pet" {



Final Status: COMPLETED SUCCESSFULLY.

2). Execute all the templates shown in the video.

Terraform Lifecycle Rule: `create_before_destroy`

Operating System: Ubuntu Linux (AWS EC2)
Terraform Version: 1.17.0
Working Directory: `/home/ubuntu/terraform-02/lab2`

---

1. Objective

The objective of this lab is to understand and implement the Terraform lifecycle rule `create_before_destroy`.

By default, Terraform may destroy an existing resource before creating its replacement.

Using `create_before_destroy = true`, Terraform attempts to create the replacement resource before destroying the existing one.

This can help reduce downtime during infrastructure replacement when both resources can temporarily exist.

2. Create Project Directory

Step 1: Create a New Directory

Executed the following commands:

```
mkdir -p ~/terraform-02/lab2
cd ~/terraform-02/lab2
```

Purpose: Create a separate working directory for the lifecycle rule demonstration without affecting Lab 1.

3. Create Terraform Configuration

Step 2: Create `main.tf`

```
cat > main.tf <<'EOF'
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
  }
}

variable "revision" {
  type    = string
  default = "v1"
}

resource "local_file" "my_pet" {
  filename = "path.module/pets −{var.revision}.txt"
  content  = "My cat name - ${var.revision}"

  lifecycle {
    create_before_destroy = true
  }
}
EOF
```

Code Explanation

- `required_providers`: Declares the HashiCorp Local provider.

- `variable "revision"`: Defines a variable with the default value `v1`.
- `local_file`: Creates a text file using Terraform.
- `filename`: Uses the revision variable to generate the filename.
- `content`: Writes the revision details into the file.
- `lifecycle`: Controls resource replacement behavior.
- `create_before_destroy = true`: Creates a replacement resource before destroying the previous resource.

4. Initialize Terraform

Step 3: Initialize, Format and Validate

terraform init
terraform fmt
terraform validate
terraform plan

Purpose:

- `terraform init` – Initializes Terraform and installs the provider.
- `terraform fmt` – Formats the configuration.
- `terraform validate` – Checks configuration validity.
- `terraform plan` – Displays the proposed resource actions.

The initial configuration uses the default revision `v1`.

5. Create the Initial Resource

Step 4: Apply Terraform

terraform apply

Enter `yes` when prompted.

Verify the initial file:

ls -l
cat pets-v1.txt

Expected content:

My cat name - v1

This represents the initial resource before replacement.

6. Demonstrate `create_before_destroy`

Step 5: Change the Revision from v1 to v2

Instead of manually editing the configuration file, pass the updated value through the Terraform command line.

terraform plan -var='revision=v2'

Purpose: Preview replacement of `pets-v1.txt` with `pets-v2.txt`.

With `create_before_destroy = true`, Terraform plans to create the new resource before destroying the old resource.

Step 6: Apply the Updated Configuration

terraform apply -var='revision=v2'

Enter:

yes

Actual Execution Output

local_file.my_pet: Creating...

local_file.my_pet: Creation complete after 0s
[id=305abfd5cb1fe7a484b814be3d5caa9712f82bb1]

local_file.my_pet (deposed object 513ed69b): Destroying...
[id=5df40df5b64ab2bba0976e77b0cc72858ef97d61]

local_file.my_pet: Destruction complete after 0s

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Result: Terraform successfully created the replacement resource first and then destroyed the previous resource.

The `deposed object` represents the old resource instance waiting to be removed after successful replacement.

7. Verify the Updated Resource

Step 7: Check the Created File

Executed:

ls -l pets-*.txt
cat pets-v2.txt
terraform state list

Actual Output

-rwxrwxr-x 1 ubuntu ubuntu 16 Oct 23 14:23 pets-v2.txt

My cat name - v2

local_file.my_pet

Verification:

- `pets-v2.txt` was created successfully.
- The updated content is `My cat name - v2`.
- The old version was removed after replacement.
- Terraform state tracks the current `local_file.my_pet` resource.

8. Understanding the Lifecycle Workflow

Without `create_before_destroy`

Existing Resource (v1)
|
v
Destroy Old Resource
|
v
Create New Resource (v2)

With `create_before_destroy = true`

Existing Resource (v1)
|
v
Create New Resource (v2)
|
v
Destroy Old Resource (v1)
|

v
New Resource (v2) Active

Key difference: The replacement is created before the old resource is destroyed.

This ordering is useful when the provider and infrastructure permit both resources to exist temporarily.

9. Real-Time DevOps Scenario

Scenario: Application Server Replacement

Consider an application running on an EC2 instance.

The company needs to replace the instance with a new version.

If the existing server is destroyed before the new server is ready, the application may experience downtime.

With a suitable infrastructure design, `create_before_destroy` can help ensure a replacement is provisioned before the old resource is removed.

Important: This lifecycle rule alone does not guarantee zero downtime. Health checks, traffic switching, resource naming, capacity and load balancer configuration must also be considered.

10. Interview Question and Answer

Q1. What is `create_before_destroy` in Terraform?

Answer: It is a Terraform lifecycle rule used to change the replacement order. When a resource needs replacement, Terraform attempts to create the new resource before destroying the existing one.

Q2. Why do we use `create_before_destroy`?

Answer: We use it to help reduce downtime during resource replacement and to maintain availability where the infrastructure supports temporary coexistence.

Q3. How do you configure `create_before_destroy`?

Answer:

```
lifecycle {
  create_before_destroy = true
```

}

Q4. What is a deposed object in Terraform?

Answer: A deposed object is an old resource instance that Terraform retains temporarily while replacing it. Terraform can destroy it after the replacement is successfully created.

11. Final Verification

Verification	Result
Terraform configuration	Completed
Lifecycle rule configured	Completed
Initial resource	v1
Updated resource	v2
New file created	`pets-v2.txt`
Previous resource destroyed	Successful
State verification	`local_file.my_pet`
Terraform apply	1 added, 1 destroyed

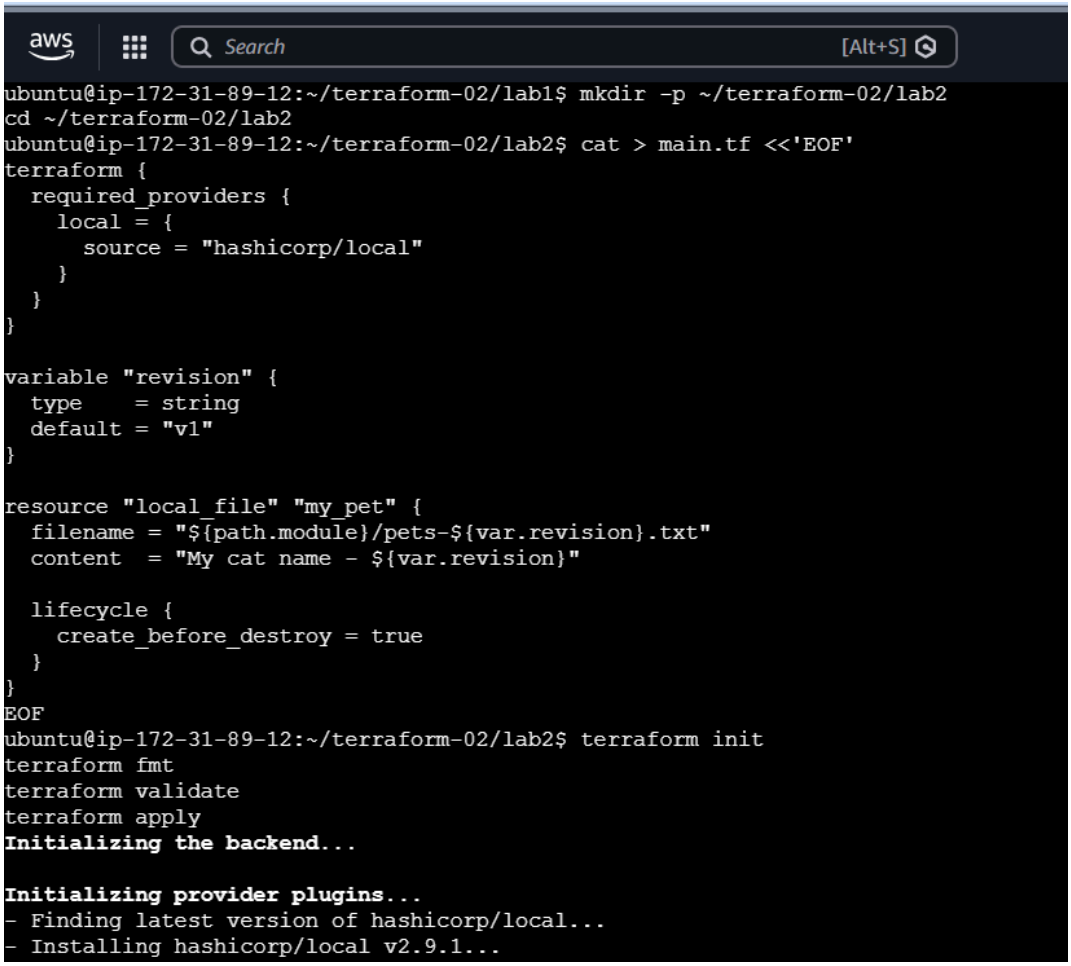
12. Conclusion

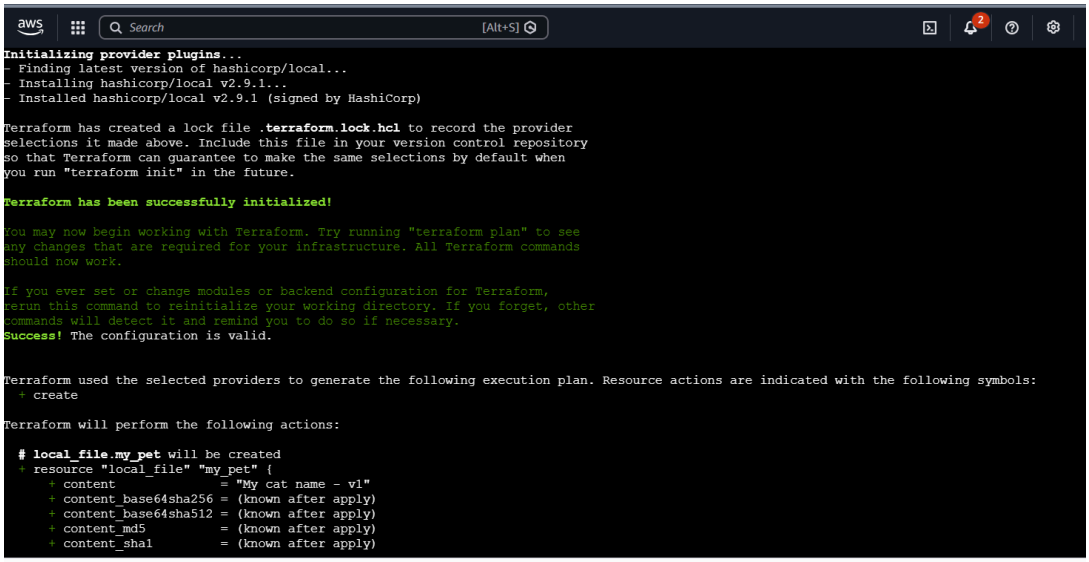
Successfully implemented and verified the Terraform `create_before_destroy` lifecycle rule.

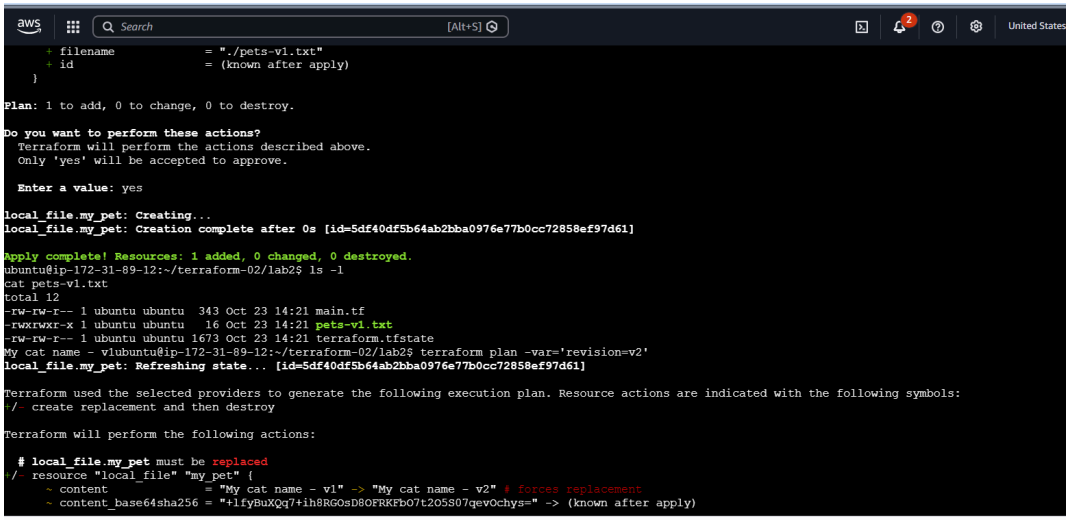
During the practical demonstration, Terraform created `pets-v2.txt` before destroying the previous resource.

The final file content was verified as `My cat name - v2`, and the managed resource was confirmed using `terraform state list`.

This lab helped me understand Terraform resource replacement and lifecycle management.


aws
Search
[Alt+S]
ubuntu@ip-172-31-89-12:~/terraform-02/lab1$ mkdir -p ~/terraform-02/lab2
cd ~/terraform-02/lab2
ubuntu@ip-172-31-89-12:~/terraform-02/lab2$ cat > main.tf <<'EOF'
terraform {
required_providers {
local = {
source = "hashicorp/local"
}
}
}
variable "revision" {
type = string
default = "v1"
}
resource "local_file" "my_pet" {
filename = "${path.module}/pets-${var.revision}.txt"
content = "My cat name - ${var.revision}"
lifecycle {
create_before_destroy = true
}
}
EOF
ubuntu@ip-172-31-89-12:~/terraform-02/lab2$ terraform init
terraform fmt
terraform validate
terraform apply
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/local...
- Installing hashicorp/local v2.9.1...


aws
Search
[Alt+S]
Initializing provider plugins...
- Finding latest version of hashicorp/local...
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
Terraform has been successfully initialized!
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
Success! The configuration is valid.
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
local_file.my_pet will be created
+ resource "local_file" "my_pet" {
+ content = "My cat name - v1"
+ content_base64sha256 = (known after apply)
+ content_base64sha512 = (known after apply)
+ content_md5 = (known after apply)
+ content_sha1 = (known after apply)



```
      + filename             = "./pets-v1.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

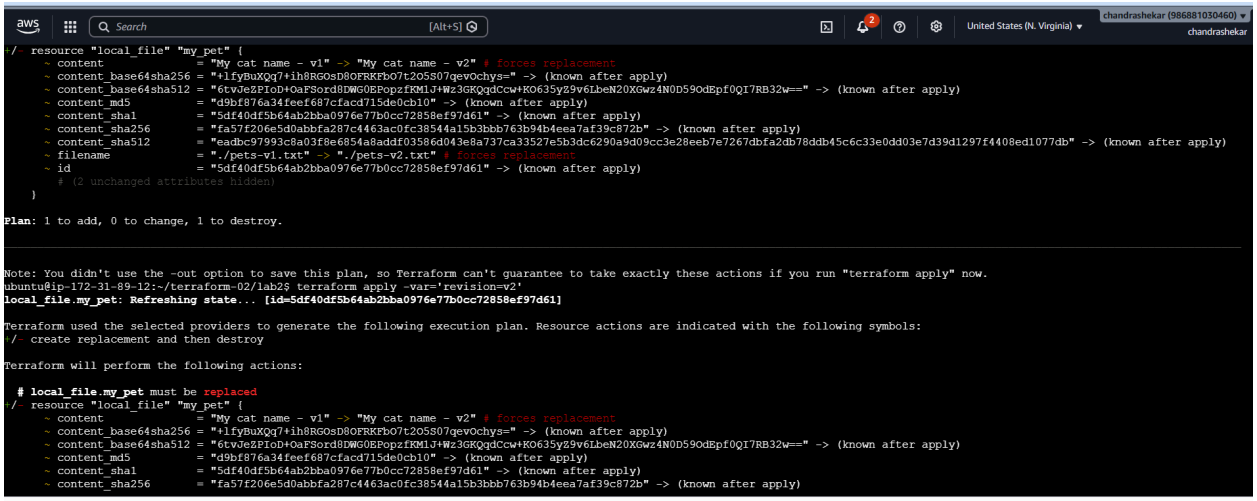
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=5df40df5b64ab2bba0976e77b0cc72858ef97d61]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
ubuntu@ip-172-31-89-12:~/terraform-02/lab2$ ls -l
cat pets-v1.txt
total 12
-rw-rw-r-- 1 ubuntu ubuntu  343 Oct 23 14:21 main.tf
-rwxrwxr-x 1 ubuntu ubuntu   16 Oct 23 14:21 pets-v1.txt
-rw-rw-r-- 1 ubuntu ubuntu 1673 Oct 23 14:21 terraform.tfstate
My cat name - v1ubuntu@ip-172-31-89-12:~/terraform-02/lab2$ terraform plan -var='revision=v2'
local_file.my_pet: Refreshing state... [id=5df40df5b64ab2bba0976e77b0cc72858ef97d61]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+/- create replacement and then destroy

Terraform will perform the following actions:

  # local_file.my_pet must be replaced
+/- resource "local_file" "my_pet" {
      ~ content              = "My cat name - v1" -> "My cat name - v2" # forces replacement
      ~ content_base64sha256 = "+lfyBuXQq7+ih8RGOsD8OFRKFbO7t2O5S07qevOchys=" -> (known after apply)
```



```
+/- resource "local_file" "my_pet" {
      ~ content              = "My cat name - v1" -> "My cat name - v2" # forces replacement
      ~ content_base64sha256 = "+lfyBuXQq7+ih8RGOsD8OFRKFbO7t2O5S07qevOchys=" -> (known after apply)
      ~ content_base64sha512 = "6tvJeZPIoD+OaFSord8DWGOEPopzfKM1J+Wz3GKQqdCcw+KO635y29v6LbeN20XGwz4N0D59OdEpf0QI7RB32w==" -> (known after apply)
      ~ content_md5          = "d9bf876a34feef687cfacd715de0cb10" -> (known after apply)
      ~ content_sha1         = "5df40df5b64ab2bba0976e77b0cc72858ef97d61" -> (known after apply)
      ~ content_sha256       = "fa57f206e5d0abbfa287c4463ac0fc38544a15b3bbb763b94b4eea7af39c872b" -> (known after apply)
      ~ content_sha512       = "eadbc97993c8a03f8e6854a8addf03586d043e8a737ca33527e5b3dc6290a9d09cc3e28eeb7e7267dbfa2db78ddb45c6c33e0dd03e7d39d1297f4408ed1077db" -> (known after apply)
      ~ filename             = "./pets-v1.txt" -> "./pets-v2.txt" # forces replacement
      ~ id                   = "5df40df5b64ab2bba0976e77b0cc72858ef97d61" -> (known after apply)
        # (2 unchanged attributes hidden)
    }

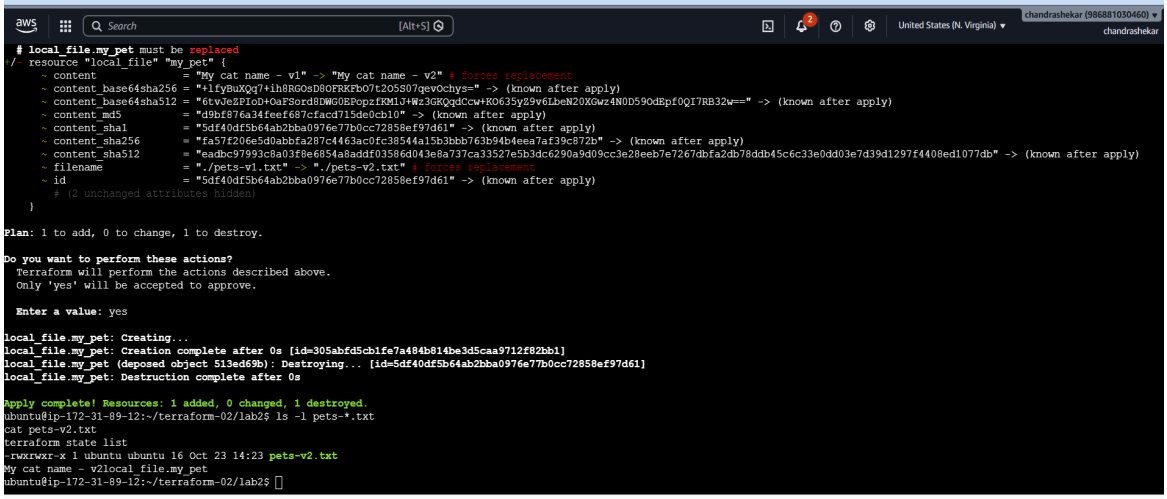
Plan: 1 to add, 0 to change, 1 to destroy.


Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
ubuntu@ip-172-31-89-12:~/terraform-02/lab2$ terraform apply -var='revision=v2'
local_file.my_pet: Refreshing state... [id=5df40df5b64ab2bba0976e77b0cc72858ef97d61]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+/- create replacement and then destroy

Terraform will perform the following actions:

  # local_file.my_pet must be replaced
+/- resource "local_file" "my_pet" {
      ~ content              = "My cat name - v1" -> "My cat name - v2" # forces replacement
      ~ content_base64sha256 = "+lfyBuXQq7+ih8RGOsD8OFRKFbO7t2O5S07qevOchys=" -> (known after apply)
      ~ content_base64sha512 = "6tvJeZPIoD+OaFSord8DWGOEPopzfKM1J+Wz3GKQqdCcw+KO635y29v6LbeN20XGwz4N0D59OdEpf0QI7RB32w==" -> (known after apply)
      ~ content_md5          = "d9bf876a34feef687cfacd715de0cb10" -> (known after apply)
      ~ content_sha1         = "5df40df5b64ab2bba0976e77b0cc72858ef97d61" -> (known after apply)
      ~ content_sha256       = "fa57f206e5d0abbfa287c4463ac0fc38544a15b3bbb763b94b4eea7af39c872b" -> (known after apply)
```



```
  # local_file.my_pet must be replaced
+/- resource "local_file" "my_pet" {
      ~ content              = "My cat name - v1" -> "My cat name - v2" # forces replacement
      ~ content_base64sha256 = "+lfyBuXQq7+ih8RGOsD8OFRKFbO7t2O5S07qevOchys=" -> (known after apply)
      ~ content_base64sha512 = "6tvJeZPIoD+OaFSord8DWGOEPopzfKM1J+Wz3GKQqdCcw+KO635y29v6LbeN20XGwz4N0D59OdEpf0QI7RB32w==" -> (known after apply)
      ~ content_md5          = "d9bf876a34feef687cfacd715de0cb10" -> (known after apply)
      ~ content_sha1         = "5df40df5b64ab2bba0976e77b0cc72858ef97d61" -> (known after apply)
      ~ content_sha256       = "fa57f206e5d0abbfa287c4463ac0fc38544a15b3bbb763b94b4eea7af39c872b" -> (known after apply)
      ~ content_sha512       = "eadbc97993c8a03f8e6854a8addf03586d043e8a737ca33527e5b3dc6290a9d09cc3e28eeb7e7267dbfa2db78ddb45c6c33e0dd03e7d39d1297f4408ed1077db" -> (known after apply)
      ~ filename             = "./pets-v1.txt" -> "./pets-v2.txt" # forces replacement
      ~ id                   = "5df40df5b64ab2bba0976e77b0cc72858ef97d61" -> (known after apply)
        # (2 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=305abfd5cb1fe7a484b814be3d5caa9712f82bb1]
local_file.my_pet (deposed object 513ed69b): Destroying... [id=5df40df5b64ab2bba0976e77b0cc72858ef97d61]
local_file.my_pet: Destruction complete after 0s

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
ubuntu@ip-172-31-89-12:~/terraform-02/lab2$ ls -l pets-*.txt
cat pets-v2.txt
terraform state list
-rwxrwxr-x 1 ubuntu ubuntu 16 Oct 23 14:23 pets-v2.txt
My cat name - v2local_file.my_pet
ubuntu@ip-172-31-89-12:~/terraform-02/lab2$
```

Final Status: COMPLETED SUCCESSFULLY.

Lab 3 – Terraform `prevent_destroy`

Lab 3: Resource Protection

Next task

Objective: Create a file using Terraform, enable `prevent_destroy`, and verify that Terraform blocks an attempted replacement.

Expected result: `Error: Instance cannot be destroyed`

Working directory: `/home/ubuntu/terraform-02/lab3`

Step 1 – Create the directory

Open your Ubuntu terminal and execute:

```
mkdir -p ~/terraform-02/lab3

cd ~/terraform-02/lab3

pwd
```

Step 2 – Create `main.tf`

Copy and paste the complete command:

```
cat > main.tf <<'EOF'

terraform {

  required_providers {

    local = {

      source = "hashicorp/local"

  }

  }

}


variable "pet_message" {
```

```
  type    = string
  default = "I love cats"
}

resource "local_file" "my_pet" {
  filename = "${path.module}/protected-pet.txt"
  content  = var.pet_message

  lifecycle {
    prevent_destroy = true
  }
}
EOF
```

Explanation: `prevent_destroy = true` prevents Terraform from approving a plan that would destroy or replace the protected resource while the lifecycle rule remains in the configuration.

Step 3 – Initialize and execute Terraform

```
terraform init

terraform fmt

terraform validate

terraform plan

terraform apply
```

When prompted, enter `yes`.

The expected first execution is:

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Step 4 – Verify the file

cat protected-pet.txt

terraform state list

Expected output:

I love cats

local_file.my_pet

Step 5 – Test `prevent_destroy`

Now imagine that someone wants to replace the existing file by changing its content.

Execute:

terraform plan -var='pet_message=I love dogs'

Expected error – This is a successful test

Error: Instance cannot be destroyed

Resource local_file.my_pet has

lifecycle.prevent_destroy set,

but the plan calls for this resource to be destroyed.

Exact wording may vary depending on the Terraform version.

The content change requires replacement for this resource, and Terraform prevents it.

Step 6 – Verify the original file is safe

cat protected-pet.txt

terraform state list

Expected result:

I love cats

local_file.my_pet

The old file remains unchanged.

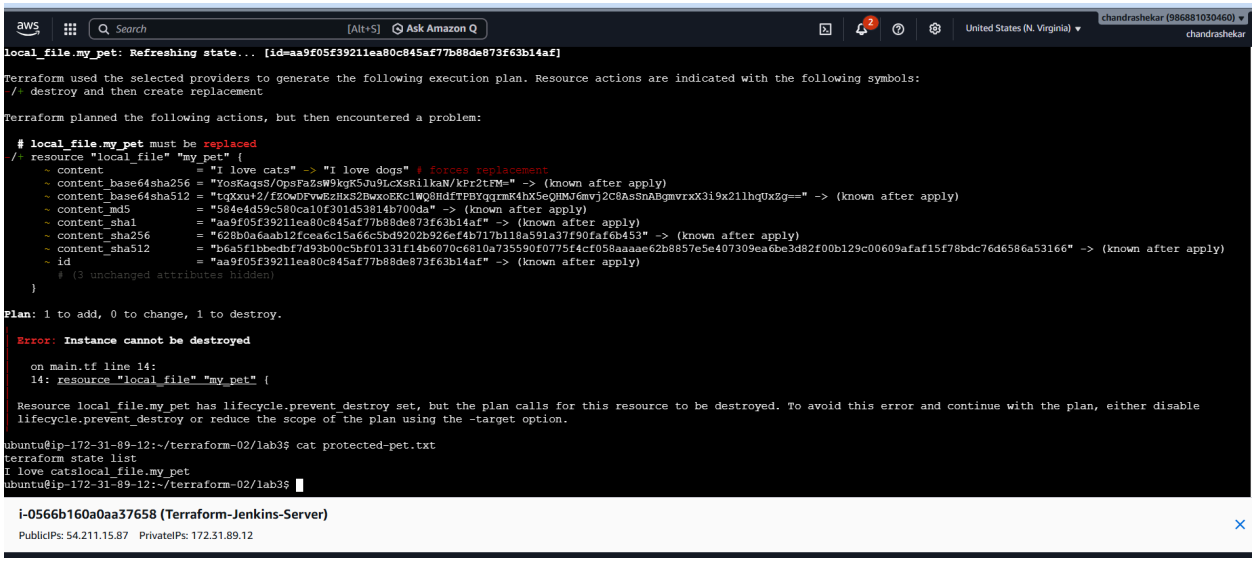
Important: Do not remove `prevent_destroy` or manually delete the file during this lab. The rule protects against Terraform-planned destruction while the resource remains configured with that rule; it does not prevent manual deletion outside Terraform.

Real-time interview example

Question: How do you prevent accidental deletion of production infrastructure using Terraform?

Answer:

"We can configure the Terraform lifecycle rule `prevent_destroy = true` on important resources. If a Terraform plan requires destroying or replacing that resource, Terraform throws an error instead of proceeding. It acts as a safety mechanism, but we should also use IAM permissions, approvals and backups for production protection."



Lab 4 – Terraform `ignore_changes`

Objective: Understand how Terraform ignores changes to selected resource attributes.

Step 1 – Create Lab 4 directory

mkdir -p ~/terraform-02/lab4

cd ~/terraform-02/lab4

Step 2 – Create `main.tf`

Copy and execute:

```
cat > main.tf <<'EOF'
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
  }
}

variable "pet_message" {
  type    = string
  default = "I love cats"
}

resource "local_file" "my_pet" {
  filename = "${path.module}/pets.txt"
  content  = var.pet_message

  lifecycle {
    ignore_changes = [content]
  }
}
```

EOF

Explanation: `ignore_changes = [content]` tells Terraform not to update the existing managed resource solely because the configured `content` value changes after creation.

Step 3 – Initialize and apply

terraform init

terraform fmt

terraform validate

terraform apply

Enter `yes` when prompted.

Verify:

cat pets.txt

terraform state list

Expected:

I love cats

local_file.my_pet

Step 4 – Test `ignore_changes`

Now pass a different value:

terraform plan -var='pet_message=I love dogs'

Even though we changed the requested content from `I love cats` to `I love dogs`, Terraform should show:

No changes. Your infrastructure matches the configuration.

Step 5 – Verify the file

terraform apply -var='pet_message=I love dogs'

cat pets.txt

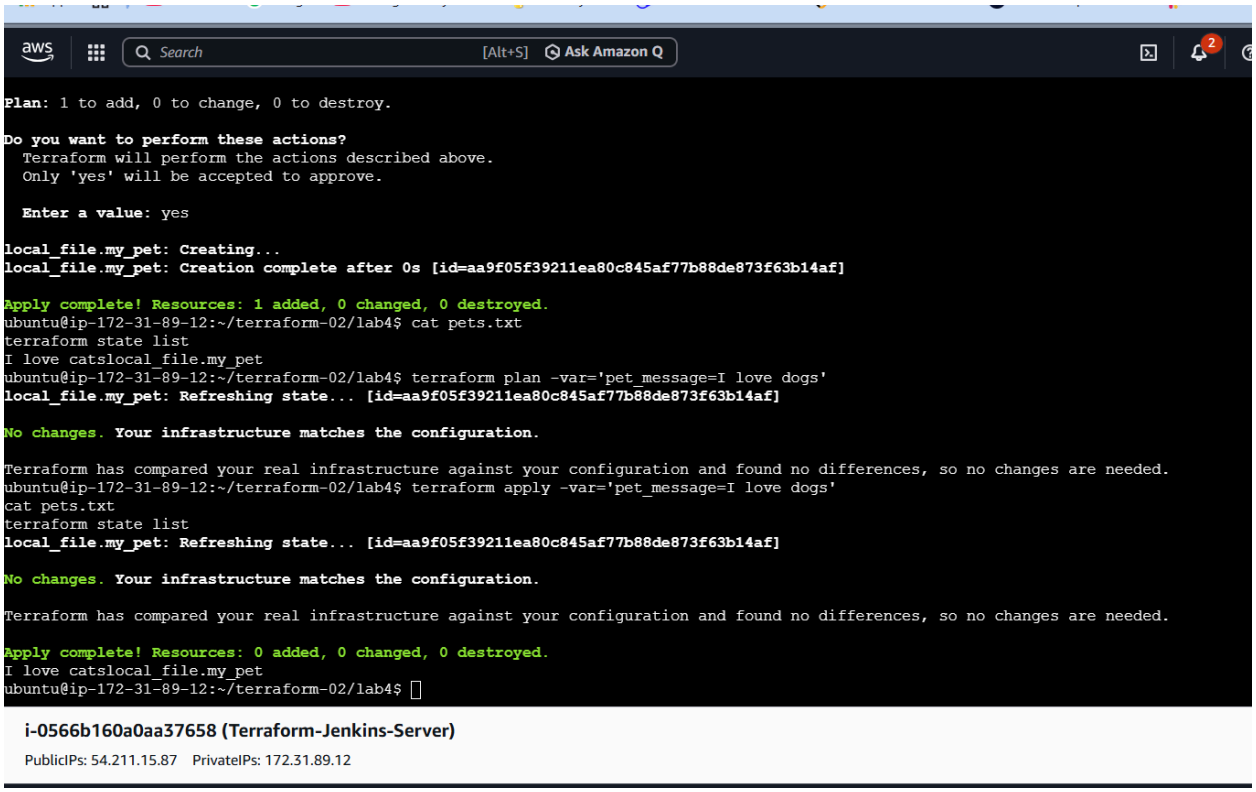
terraform state list

Expected file content:

I love cats

Why? Because `ignore_changes` prevents Terraform from acting on this change to the existing resource's `content` attribute.

Note that the initial creation still uses the configured value, and `ignore_changes` does not prevent all other resource changes.



Lab 5: Terraform Variables

In your Terraform-02 video, the trainer demonstrates different methods of using variables. We'll practice three methods:

1. **Variables with default values in `variables.tf`.**
2. **Variables entered interactively at runtime.**
3. **Variables passed directly using `-var`.**

Step 1 – Create Lab 5 directory

```
mkdir -p ~/terraform-02/lab5

cd ~/terraform-02/lab5

pwd
```

Step 2 – Create `main.tf`

```
cat > main.tf <<'EOF'

terraform {

  required_providers {

    local = {

      source = "hashicorp/local"

    }

    random = {

      source = "hashicorp/random"

    }

  }

}


resource "local_file" "my_pet" {

  filename = "path.module/{var.filename}"

  content  = var.content

}


resource "random_pet" "petname" {

  prefix    = var.prefix

  separator = "."
```

```
  length    = var.length
}


output "pet_name" {
  value = random_pet.petname.id
}
EOF
```

Step 3 – Create `variables.tf`

```
cat > variables.tf <<'EOF'
variable "filename" {
  description = "Name of the local file"
  type        = string
  default     = "pets.txt"
}


variable "content" {
  description = "Content of the file"
  type        = string
  default     = "I love dogs"
}


variable "prefix" {
  description = "Prefix for random pet name"
  type        = string
```

```
  default     = "Mr"
}

variable "length" {
  description = "Number of words in pet name"
  type        = number
  default     = 1
}
EOF
```

Step 4 – Understand these variable types

Variable	Type	Default value
`filename`	string	`pets.txt`
`content`	string	`I love dogs`
`prefix`	string	`Mr`
`length`	number	`1`

Important: The main configuration uses `var.filename`, `var.content`, `var.prefix`, and `var.length` to reference the declared variables.

Step 5 – Initialize and execute

Run these commands:

terraform init

terraform fmt

terraform validate

terraform plan

terraform apply

Type `yes` when prompted.

Expected plan:

Plan: 2 to add, 0 to change, 0 to destroy.

Step 6 – Verify results

terraform state list

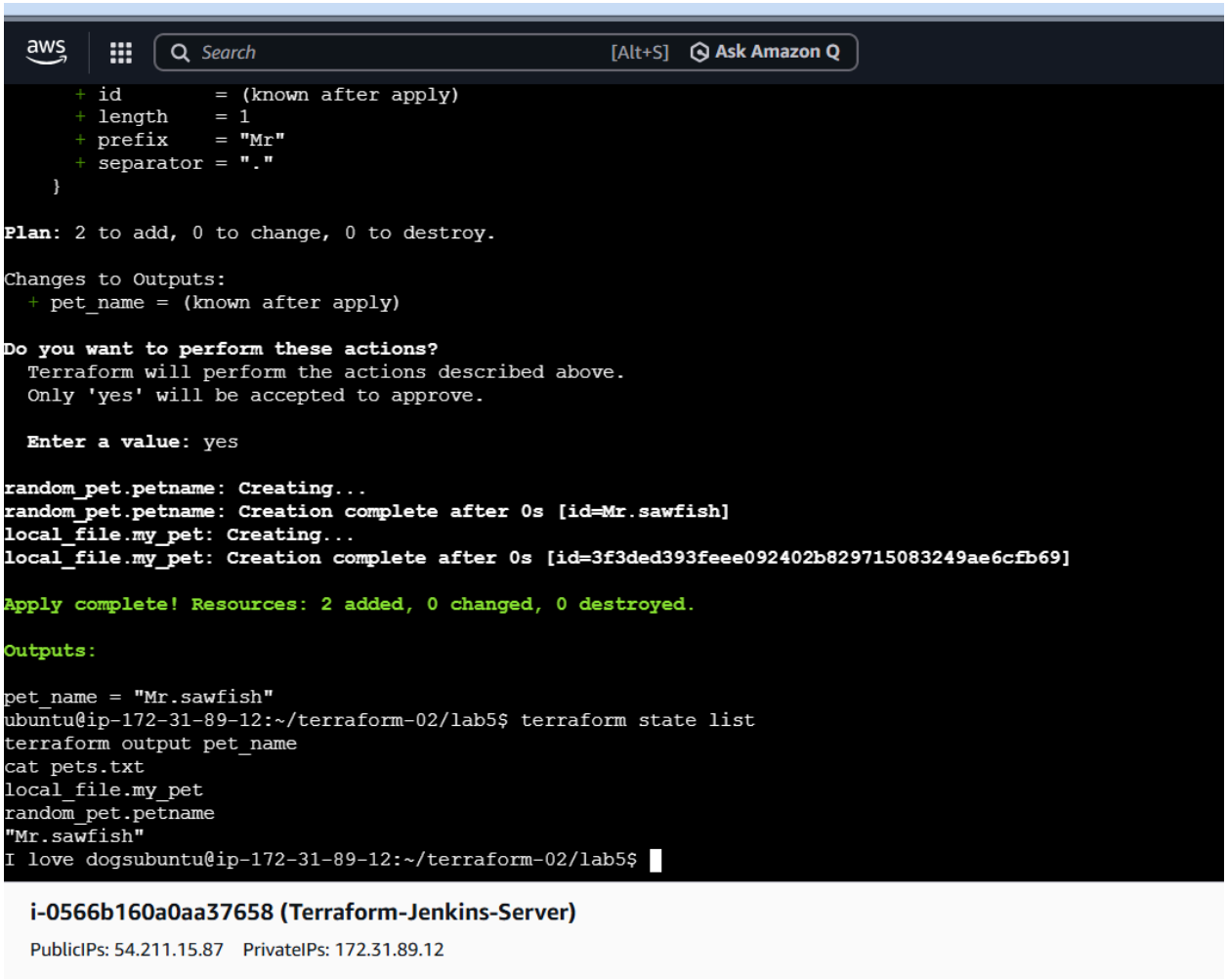
terraform output pet_name

cat pets.txt

Expected file content:

I love dogs

Your random pet name will be different.



Lab 6: Terraform Runtime Variables

In your trainer's Terraform-02 video, two runtime input methods are demonstrated:

1. Enter variable values interactively when Terraform asks.
2. Pass variable values directly using the `-var` option.

We'll practice both.

Step 1 – Create Lab 6 directory

Since Lab 5 is already completed, reuse its `main.tf` in a new folder.

mkdir -p ~/terraform-02/lab6
cd ~/terraform-02/lab6

cp ../lab5/main.tf .

ls -l

This copies only `main.tf`, not the Terraform state.

Step 2 – Create `variables.tf`

For this exercise, `filename` and `content` have no default values. Terraform will ask for them at runtime.

```
cat > variables.tf <<'EOF'
variable "filename" {
  description = "Enter the file name"
  type        = string
}

variable "content" {
  description = "Enter the file content"
  type        = string
}

variable "prefix" {
  type    = string
  default = "Mr"
}

variable "length" {
  type    = number
  default = 1
}
EOF
```

Step 3 – Initialize Terraform

```
terraform init
terraform fmt
terraform validate
```

Expected result:

```
Terraform has been successfully initialized!
Success! The configuration is valid.
```

Step 4 – Method 1: Enter variables interactively

Run:

terraform apply

Terraform will request the missing values.

Example input

```
var.content
  Enter the file content

  Enter a value: I love cats

var.filename
  Enter the file name

  Enter a value: pets.txt
```

The order of prompts may vary. Enter each value against the matching variable name.

Terraform will show the execution plan. Type `yes` to approve it.

Verify Method 1

```
cat pets.txt
echo
terraform state list
terraform output pet_name
```

Expected file content:

I love cats

Step 5 – Method 2: Pass variables using `-var`

Now we'll change the filename and content directly through the command line.

Execute:

```
terraform plan \
  -var='filename=dog.txt' \
  -var='content=I love Terraform'
```

This should plan the replacement of the local file.

If the plan is correct, run:

```
terraform apply \
  -var='filename=dog.txt' \
  -var='content=I love Terraform'
```

Enter `yes` when prompted.

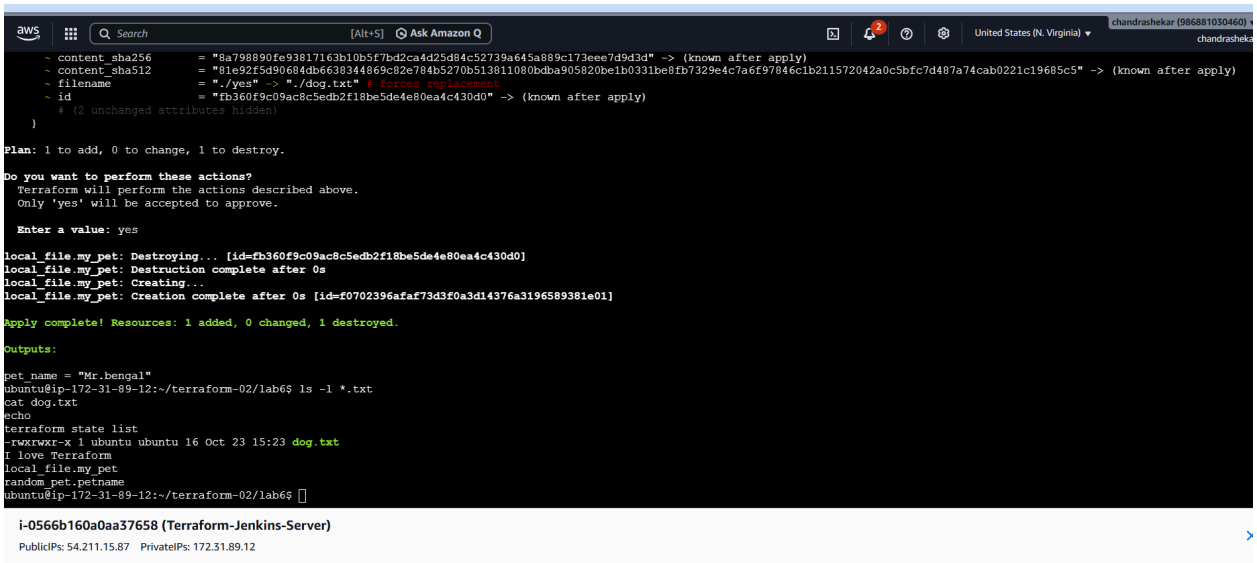
Step 6 – Verify Method 2

ls -l *.txt
cat dog.txt
echo
terraform state list

Expected:

I love Terraform

local_file.my_pet
Random_pet.petname



Important interview question

Q: How many ways can we pass variables in Terraform?

Answer: Terraform supports multiple methods, including defaults in variable declarations, interactive input, `-var` command-line arguments, `.tfvars` files, automatically loaded variable files, and `TF_VAR_` environment variables.

Lab 7: Terraform Environment Variables

This is the next variable method introduced near the end of your Terraform-02 video.

Goal: Pass Terraform input variables using Linux `export` commands, then understand how command-line `-var` values override environment values.

Step 1 – Create Lab 7 directory

We'll reuse your working Lab 5 configuration without copying its state.

mkdir -p ~/terraform-02/lab7

cd ~/terraform-02/lab7

cp ../lab5/main.tf .

cp ../lab5/variables.tf .

ls -l

Step 2 – Export Terraform variables

Run:

export TF_VAR_filename="env-pets.txt"

export TF_VAR_content="I love environment variables"

export TF_VAR_prefix="DevOps"

export TF_VAR_length=2

Terraform reads input variables with the `TF_VAR_` prefix from the environment. These values override defaults declared in `variables.tf`.

Step 3 – Initialize and apply

terraform init

terraform fmt

terraform validate

terraform plan

terraform apply

Enter `yes` if the plan is correct.

Step 4 – Verify results

cat env-pets.txt

echo

terraform output pet_name

terraform state list

Expected file content:

I love environment variables

The generated pet name should have the `DevOps` prefix and two randomly selected words.

Step 5 – Test variable precedence

Now override the environment variables using `-var`:

```
terraform plan \
  -var='filename=cli-pets.txt' \
  -var='content=CLI overrides environment'
```

You should see a plan to replace the existing local file. The random pet resource should remain unchanged because its inputs have not changed.

If the plan is correct:

terraform apply \

  -var='filename=cli-pets.txt' \

  -var='content=CLI overrides environment'

Enter `yes`, then verify:

cat cli-pets.txt

echo

terraform state list

Expected content:

CLI overrides environment

This demonstrates that `-var` has higher precedence than `TF_VAR_` environment variables.

HashiCorp Developer

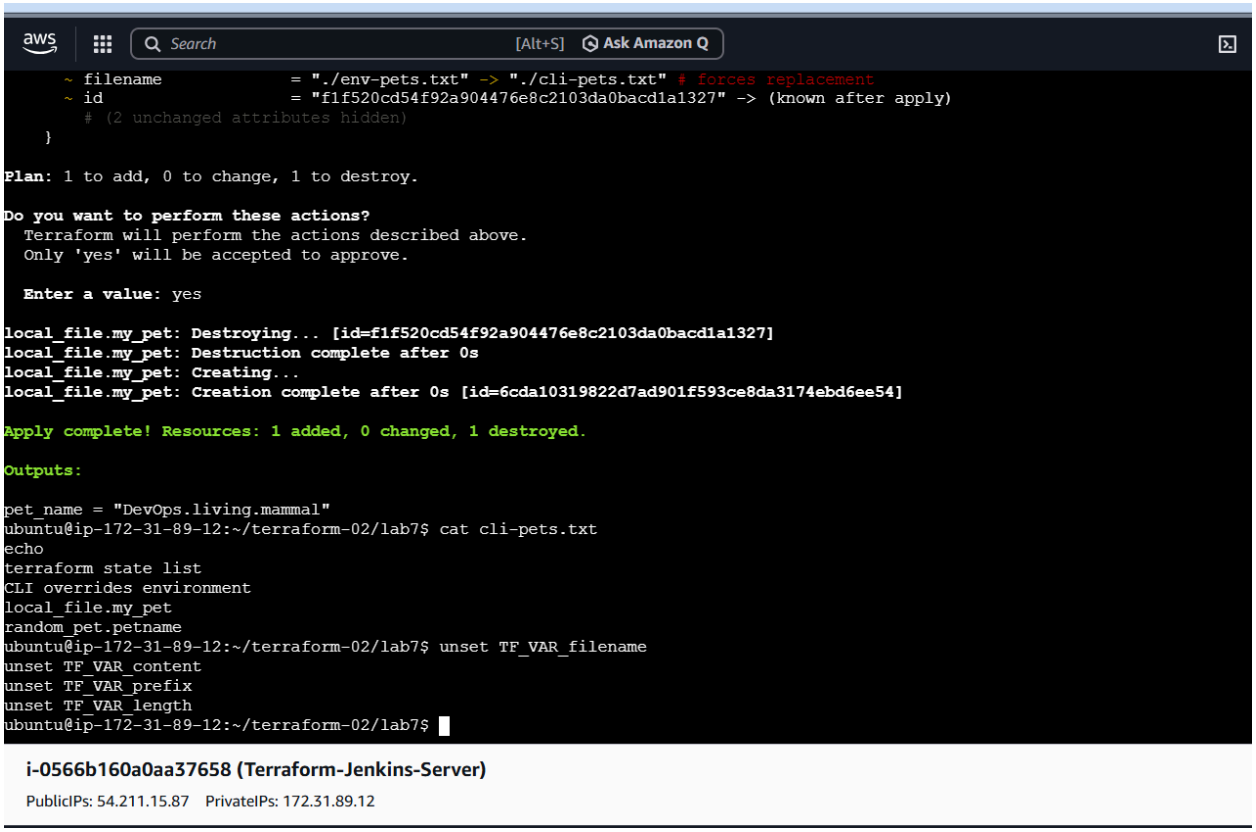
Step 6 – Clear temporary environment variables

unset TF_VAR_filename

unset TF_VAR_content

unset TF_VAR_prefix

unset TF_VAR_length



3). Integrate **Terraform in Jenkins** using the **Terraform plugin**.

TERRAFORM INTEGRATION WITH JENKINS

Jenkins Terraform Plugin Configuration

Objective

Integrate Terraform with Jenkins using the Terraform plugin and execute a Terraform configuration through a CI/CD pipeline.

Step 1 – Verify Terraform Installation

Execute on Ubuntu:

command -v terraform

```
dirname "$(command -v terraform)"
sudo -u jenkins terraform version
```

Actual output:

```
/usr/bin/terraform
/usr/bin

Terraform v1.17.0
on linux_amd64
```

This confirmed that the Jenkins service user could execute Terraform.

Step 2 – Install Terraform Plugin

1. Open Jenkins Dashboard.
2. Navigate to **Manage Jenkins → Plugins**.
3. Select **Available plugins**.
4. Search for **Terraform**.
5. Install the Terraform plugin.
6. Verify installation under **Installed plugins**.

Step 3 – Configure Terraform Installation

Navigate to:

Manage Jenkins → Tools → Terraform installations

Configured the following:

Configuration	Value
Name	Terraform-CLI
Install automatically	Disabled
Installation directory	/usr/bin

Save the configuration.

This allows Jenkins to use the existing Terraform installation.

11. Create Jenkins Pipeline Job

Step 1 – Create New Pipeline

Navigate to:

Jenkins Dashboard → New Item

Entered:

Job Name: terraform-02-pipeline
Job Type: Pipeline

Selected **Pipeline script** under the Pipeline definition.

Step 2 – Verify Jenkins Terraform Tool

The first test pipeline resolved the Terraform installation using the plugin's tool type:

```
pipeline {
    agent any

    stages {
        stage('Check Terraform Installation') {
            steps {
                script {
                    def tfHome = tool(
                        name: 'Terraform-CLI',
                        type: 'org.jenkinsci.plugins.terraform.TerraformInstallation'
                    )

                    withEnv(["PATH+TERRAFORM=${tfHome}"]) {
                        sh 'terraform version'
                        sh 'which terraform'
                    }
                }
            }
        }

        stage('Check Jenkins User') {
            steps {
                sh 'whoami'
                sh 'pwd'
            }
        }
    }
```

}

Actual verification:

Terraform v1.17.0
on linux_amd64

/usr/bin/terraform

jenkins

/var/lib/jenkins/workspace/terraform-02-pipeline

Finished: SUCCESS

Result: The Jenkins Terraform tool integration test passed.

12. Implement Complete Terraform Jenkins Pipeline

Updated the existing Pipeline script with the following stages.

Jenkinsfile

```
pipeline {
    agent any

    parameters {
        booleanParam(
            name: 'APPLY_CHANGES',
            defaultValue: false,
            description: 'Apply Terraform after reviewing the plan'
        )
    }

    environment {
        TF_IN_AUTOMATION = 'true'
    }

    stages {
        stage('Check Terraform Plugin') {
            steps {
```

```
                script {
                    env.TF_HOME = tool(
                        name: 'Terraform-CLI',
                        type: 'org.jenkinsci.plugins.terraform.TerraformInstallation'
                    )
                }
                sh '"$TF_HOME/terraform" version'
            }
        }

        stage('Create Terraform File') {
            steps {
                writeFile file: 'main.tf', text: '''
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
    random = {
      source = "hashicorp/random"
    }
  }
}

resource "local_file" "my_pet" {
  filename = "${path.module}/pets.txt"
  content  = "I love cats"
}

resource "random_pet" "my_pet" {
  prefix    = "Mr"
  separator = "."
  length    = 1
}

output "pet_name" {
  value = random_pet.my_pet.id
}
'''
            }
        }

        stage('Terraform Init') {
            steps {
```

```
            sh '"$TF_HOME/terraform" init -input=false'
        }
    }

    stage('Terraform Validate') {
        steps {
            sh '"$TF_HOME/terraform" fmt -check'
            sh '"$TF_HOME/terraform" validate'
        }
    }

    stage('Terraform Plan') {
        steps {
            sh '"$TF_HOME/terraform" plan -input=false -out=tfplan'
        }
    }

    stage('Terraform Apply') {
        when {
            expression { params.APPLY_CHANGES }
        }
        steps {
            input message: 'Apply the Terraform plan?',
                ok: 'Apply'

            sh '"$TF_HOME/terraform" apply -input=false tfplan'
        }
    }

    stage('Verify Resources') {
        when {
            expression { params.APPLY_CHANGES }
        }
        steps {
            sh '''
                "$TF_HOME/terraform" state list
                "$TF_HOME/terraform" output pet_name
                cat pets.txt
                grep -qx "I love cats" pets.txt
            '''
        }
    }
}
```

```
    post {
        success {
            echo 'Terraform Pipeline completed successfully!'
        }
        failure {
            echo 'Terraform Pipeline failed. Check Console Output.'
        }
    }
}
```

13. Execute Jenkins Pipeline

Step 1 – Run Build #2 (Plan Only)

Executed the Pipeline with:

APPLY_CHANGES = false

Jenkins successfully completed the Terraform initialization, validation and plan stages.

Actual output:

Terraform has been successfully initialized!

Success! The configuration is valid.

Plan: 2 to add, 0 to change, 0 to destroy.

Terraform Apply: Skipped
Verify Resources: Skipped

Finished: SUCCESS

Step 2 – Run Build #3 (Terraform Apply)

Selected **Build with Parameters**.

Configured:

APPLY_CHANGES = true

Started the build.

Jenkins executed:

1. Check Terraform Plugin
2. Create Terraform File
3. Terraform Init
4. Terraform Validate
5. Terraform Plan
6. Terraform Apply
7. Verify Resources

Step 3 – Manual Approval

The Jenkins Pipeline paused with the message:

Apply the Terraform plan?

The deployment was approved through Jenkins by the admin user.

Actual log:

Pausing
Resuming
Approved by admin

Step 4 – Terraform Apply Execution

Jenkins executed:

/usr/bin/terraform apply -input=false tfplan

Actual output:

random_pet.my_pet: Creating...
random_pet.my_pet: Creation complete
[id=Mr.egret]

local_file.my_pet: Creating...
local_file.my_pet: Creation complete

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

pet_name = "Mr.egret"

Step 5 – Resource Verification

Jenkins automatically executed:

terraform state list
terraform output pet_name
cat pets.txt

Actual output:

local_file.my_pet
random_pet.my_pet

"Mr.egret"

I love cats

Final Jenkins Pipeline Output

Terraform Pipeline completed successfully!

Finished: SUCCESS

Result: Jenkins Terraform Pipeline successfully completed.

14. Verify Resources from Ubuntu

The following commands can be used to independently verify the resources created by Jenkins:

```
sudo -u jenkins bash -c '
cd /var/lib/jenkins/workspace/terraform-02-pipeline
terraform state list
terraform output pet_name
cat pets.txt
'
```

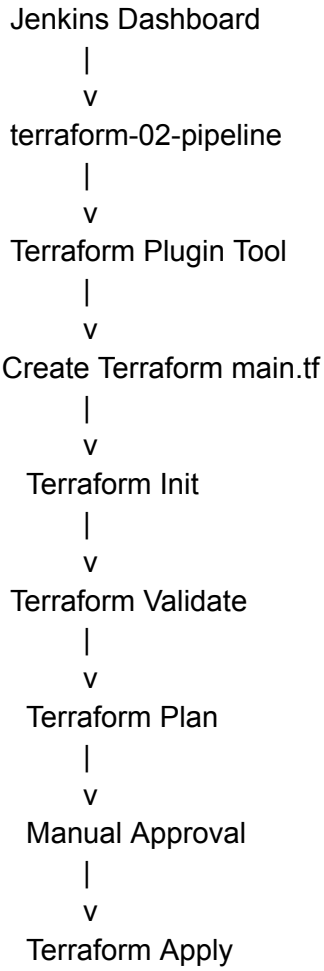
The Jenkins console log already confirmed successful resource verification.

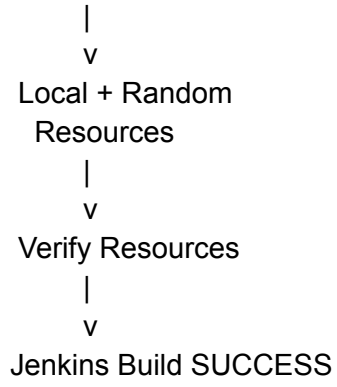
15. Jenkins Pipeline Stage Explanation

Stage	Purpose

Check Terraform Plugin	Resolves the configured Terraform installation
Create Terraform File	Generates main.tf in the Jenkins workspace
Terraform Init	Initializes Terraform and provider dependencies
Terraform Validate	Checks formatting and configuration validity
Terraform Plan	Creates a saved execution plan
Manual Approval	Requires confirmation before applying changes
Terraform Apply	Executes the saved Terraform plan
Verify Resources	Checks Terraform state, output and file contents

Terraform Jenkins Architecture





Important Learning Outcomes

Through the Terraform-02 assignment, I learned:

1. How Terraform uses providers to create and manage resources.
2. How Terraform state stores information about managed infrastructure.
3. How lifecycle rules control resource replacement and destruction.
4. How variables make Terraform configurations reusable.
5. How to pass variables interactively, using CLI arguments and environment variables.
6. How variable precedence works.
7. How to configure the Terraform plugin in Jenkins.
8. How to create Terraform files automatically inside a Jenkins Pipeline.
9. How to implement Init, Validate, Plan and Apply stages.
10. How to add manual approval before deploying infrastructure.
11. How to verify resources through Terraform state and outputs.

Real-Time Interview Questions and Answers

Q1. What is Terraform state?

Terraform state stores mappings and attributes of Terraform-managed resources, helping Terraform calculate infrastructure changes.

Q2. What is create_before_destroy?

It instructs Terraform to create a replacement resource before destroying the old resource when replacement is required.

Q3. What is prevent_destroy?

It blocks Terraform plans that would destroy or replace a protected resource while the lifecycle rule remains configured.

Q4. What is ignore_changes?

It tells Terraform not to reconcile changes to selected resource attributes after creation.

Q5. What is the purpose of variables.tf?

It declares input variables that make configurations reusable and configurable.

Q6. How do you pass variables through the command line?

Using the `-var` flag:

terraform apply -var='filename=dog.txt'

Q7. What is the purpose of TF_VAR_?

Terraform reads environment variables beginning with `TF_VAR_` as input variable values.

Q8. Why integrate Terraform with Jenkins?

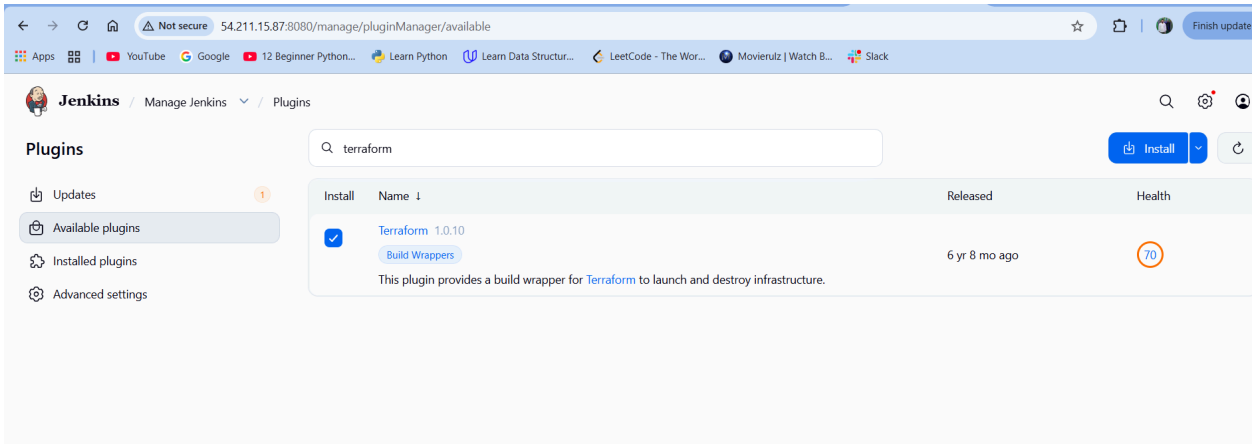
To automate infrastructure validation, planning, deployment and verification through repeatable CI/CD workflows.

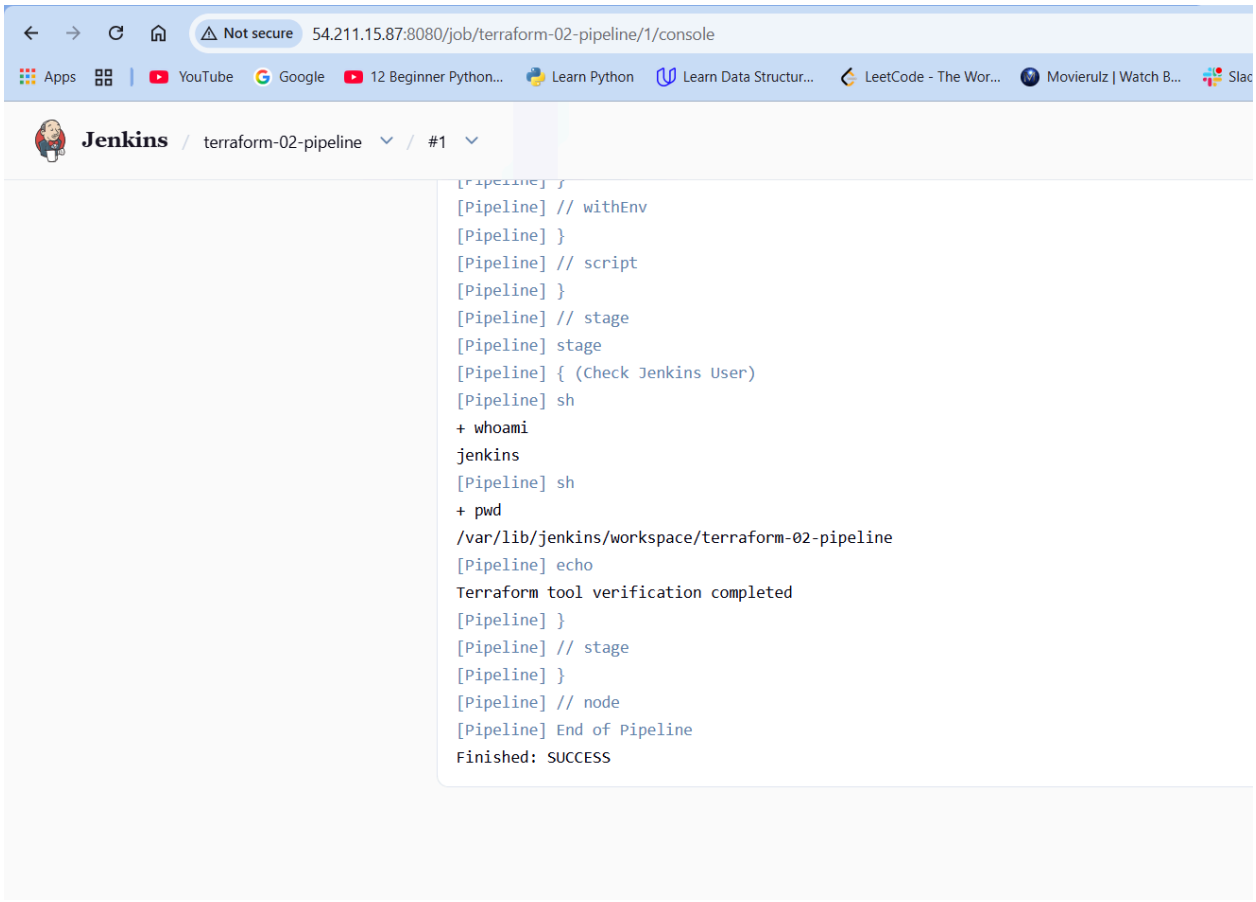
Q9. Why do we use a manual approval stage?

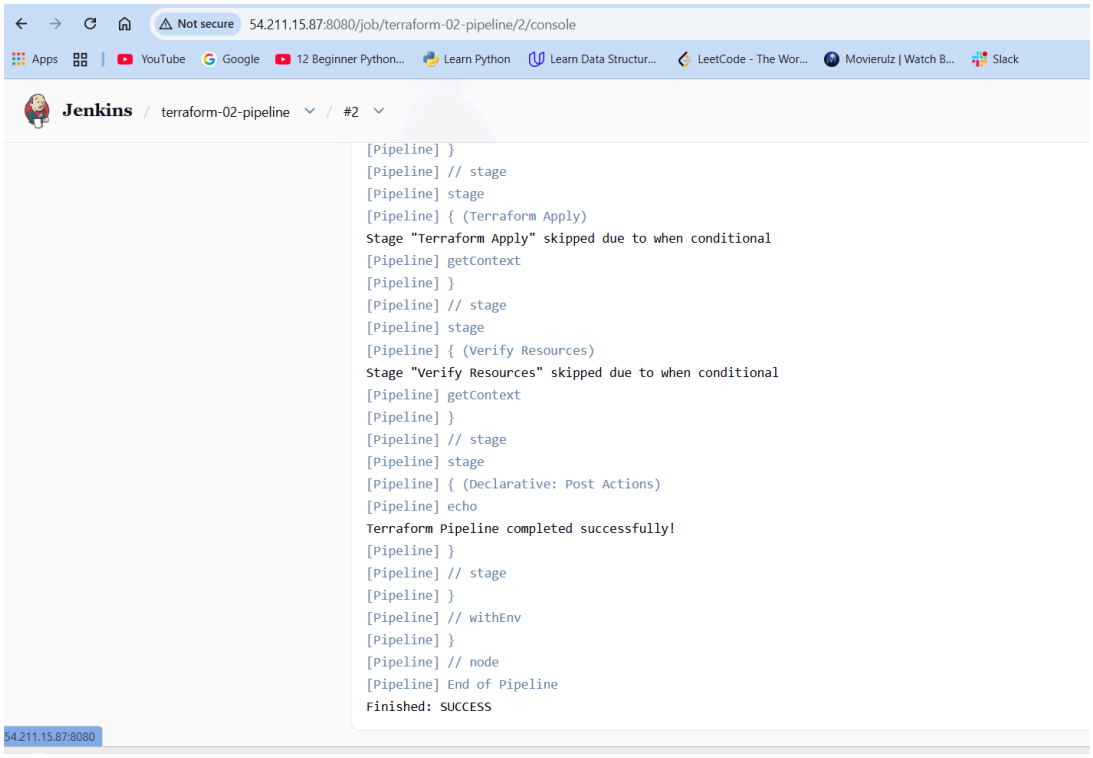
To review infrastructure changes before execution and reduce the risk of accidental modifications.

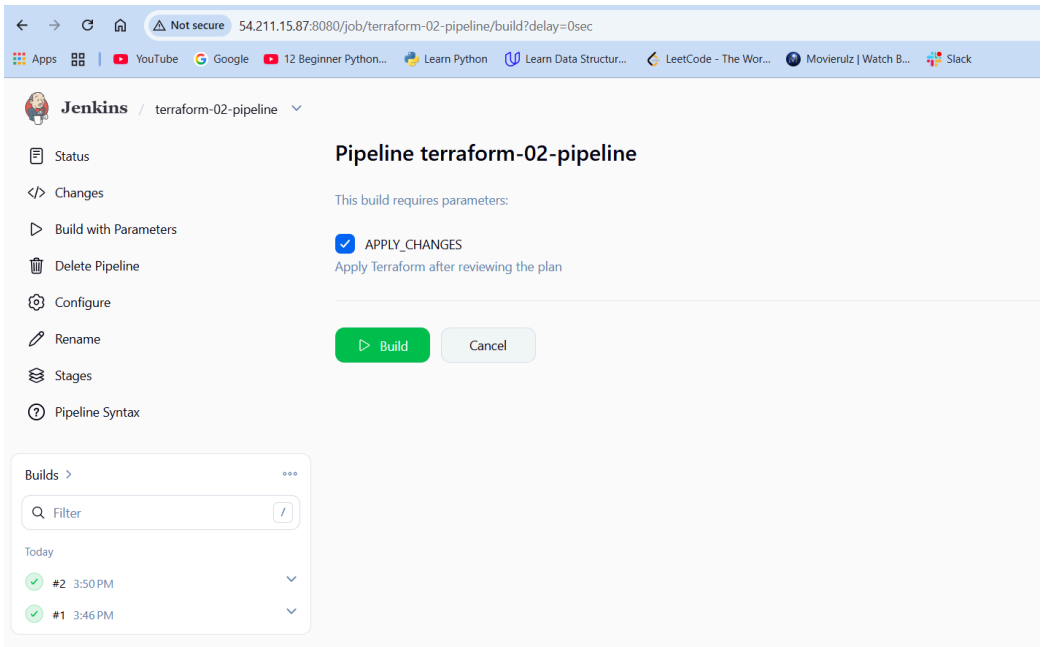
Q10. What happens when a Jenkins Terraform deployment fails?

The pipeline reports failure. Engineers inspect the stage logs, investigate the error, verify the infrastructure and state, and retry only after correcting the issue saf

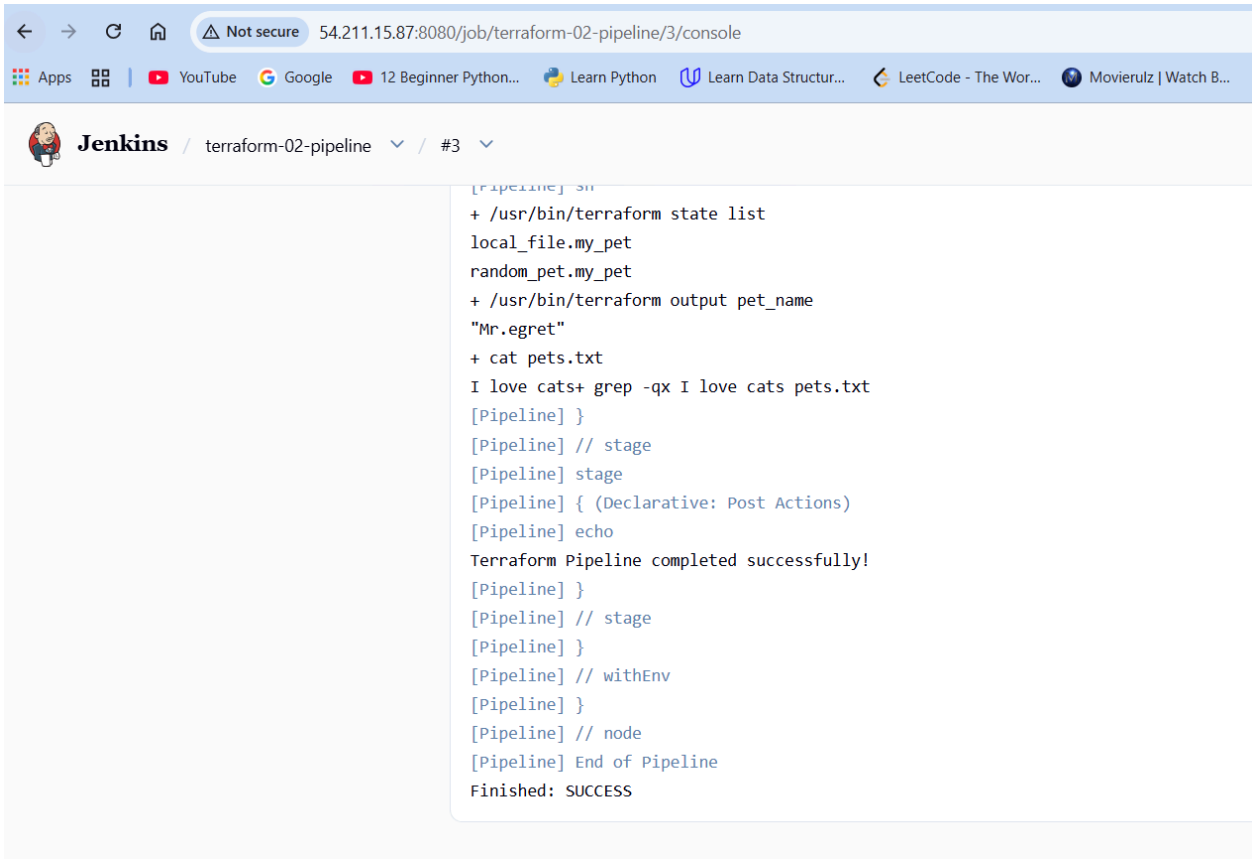

Not secure 54.211.15.87:8080/manage/pluginManager/available
Jenkins / Manage Jenkins / Plugins
Plugins
terraform
Install
Updates
Available plugins
Installed plugins
Advanced settings
Install
Name
Released
Health
Terraform 1.0.10
Build Wrappers
6 yr 8 mo ago
70
This plugin provides a build wrapper for Terraform to launch and destroy infrastructure.

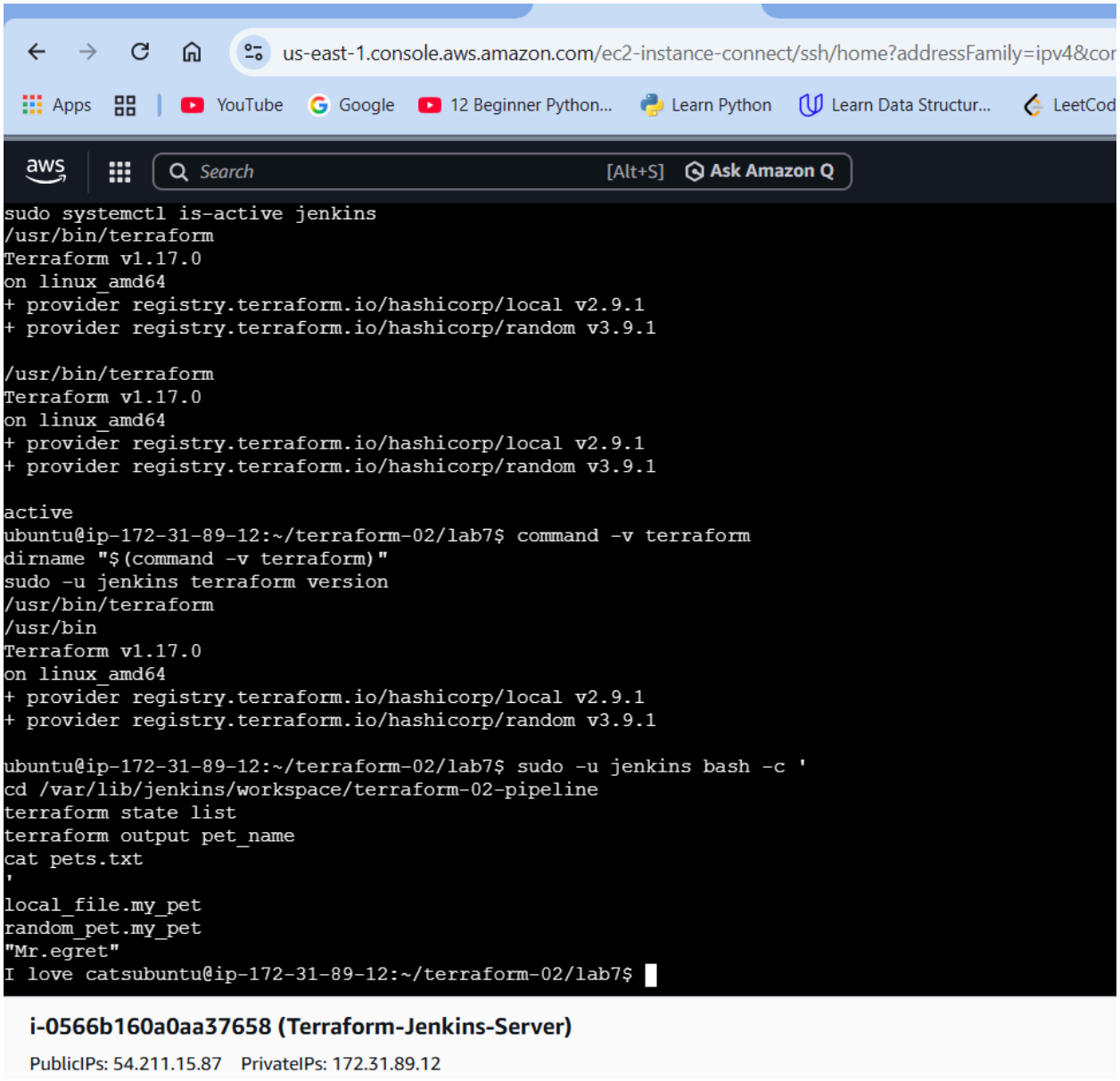

Not secure 54.211.15.87:8080/job/terraform-02-pipeline/1/console
Jenkins / terraform-02-pipeline / #1
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Check Jenkins User)
[Pipeline] sh
+ whoami
jenkins
[Pipeline] sh
+ pwd
/var/lib/jenkins/workspace/terraform-02-pipeline
[Pipeline] echo
Terraform tool verification completed
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Not secure 54.211.15.87:8080/job/terraform-02-pipeline/2/console
Apps YouTube Google 12 Beginner Python... Learn Python Learn Data Structur... LeetCode - The Wor... Movierulz | Watch B... Slack
Jenkins / terraform-02-pipeline / #2
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Terraform Apply)
Stage "Terraform Apply" skipped due to when conditional
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Resources)
Stage "Verify Resources" skipped due to when conditional
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Terraform Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
54.211.15.87:8080


Not secure 54.211.15.87:8080/job/terraform-02-pipeline/build?delay=0sec
Apps YouTube Google 12 Beginner Python... Learn Python Learn Data Structur... LeetCode - The Wor... Movierulz | Watch B... Slack
Jenkins / terraform-02-pipeline
Status
Changes
Build with Parameters
Delete Pipeline
Configure
Rename
Stages
Pipeline Syntax
Builds
Filter
Today
#2 3:50 PM
#1 3:46 PM
Pipeline terraform-02-pipeline
This build requires parameters:
APPLY_CHANGES
Apply Terraform after reviewing the plan
Build
Cancel


Not secure 54.211.15.87:8080/job/terraform-02-pipeline/3/console
Apps YouTube Google 12 Beginner Python... Learn Python Learn Data Structur... LeetCode - The Wor... Movierulz | Watch B... Slack
Jenkins / terraform-02-pipeline / #3
[1m # random_pet.my_pet[0m will be created
[0m [32m+[0m[0m resource "random_pet" "my_pet" {
[32m+[0m[0m id = (known after apply)
[32m+[0m[0m length = 1
[32m+[0m[0m prefix = "Mr"
[32m+[0m[0m separator = "."
}
[1mPlan:[0m [0m2 to add, 0 to change, 0 to destroy.
Changes to Outputs:
[32m+[0m[0m pet_name = (known after apply)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Terraform Apply)
[Pipeline] input
Apply the Terraform plan?
Apply or Abort


Not secure 54.211.15.87:8080/job/terraform-02-pipeline/3/console
Apps YouTube Google 12 Beginner Python... Learn Python Learn Data Structur... LeetCode - The Wor... Movierulz | Watch B...
Jenkins / terraform-02-pipeline / #3
+ /usr/bin/terraform state list
local_file.my_pet
random_pet.my_pet
+ /usr/bin/terraform output pet_name
"Mr.egret"
+ cat pets.txt
I love cats+ grep -qx I love cats pets.txt
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Terraform Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&cor
Apps
YouTube
Google
12 Beginner Python...
Learn Python
Learn Data Structur...
LeetCod
aws
Search
[Alt+S]
Ask Amazon Q
sudo systemctl is-active jenkins
/usr/bin/terraform
Terraform v1.17.0
on linux_amd64
+ provider registry.terraform.io/hashicorp/local v2.9.1
+ provider registry.terraform.io/hashicorp/random v3.9.1
/usr/bin/terraform
Terraform v1.17.0
on linux_amd64
+ provider registry.terraform.io/hashicorp/local v2.9.1
+ provider registry.terraform.io/hashicorp/random v3.9.1
active
ubuntu@ip-172-31-89-12:~/terraform-02/lab7$ command -v terraform
dirname "$(command -v terraform)"
sudo -u jenkins terraform version
/usr/bin/terraform
/usr/bin
Terraform v1.17.0
on linux_amd64
+ provider registry.terraform.io/hashicorp/local v2.9.1
+ provider registry.terraform.io/hashicorp/random v3.9.1
ubuntu@ip-172-31-89-12:~/terraform-02/lab7$ sudo -u jenkins bash -c '
cd /var/lib/jenkins/workspace/terraform-02-pipeline
terraform state list
terraform output pet_name
cat pets.txt
'
local_file.my_pet
random_pet.my_pet
"Mr.egret"
I love catsubuntu@ip-172-31-89-12:~/terraform-02/lab7$
i-0566b160a0aa37658 (Terraform-Jenkins-Server)
PublicIPs: 54.211.15.87 PrivateIPs: 172.31.89.12